



Lecture 2

Relational Algebra

CSC1022 Introduction to Databases

CSC1005 Data Science & Databases
2025/2026 Semester 2

Prof. Mark Roantree
Uttaran Bera
School of Computing
@ Dublin City University



1 [The Relational Algebra: An Overview](#)

2 [Unary Operations](#)

3 [Set Operations](#)

4 [Join Operations](#)

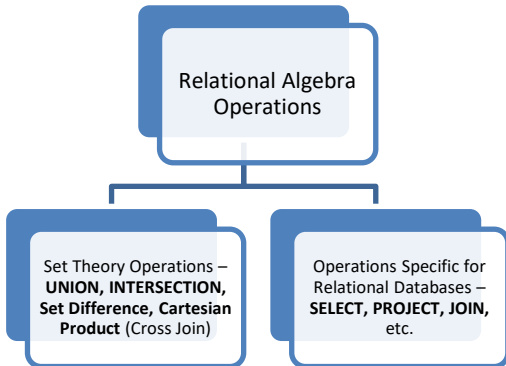


- Does the Relational Model support query operations?
- My dataset is very large, I need to extract a subset for Machine Learning (selection, projection)
- My dataset is separated, can I generate a single dataset? (Product, Join)



- Unlike SQL which is a *practical* language, the relational algebra is a *formal* language. It provides the foundation for the SQL query language.
- In the previous section, we presented the *structures* and *constraints* of the formal **relational model**.
- The basic set of relational model *operations* is the **relational algebra**.

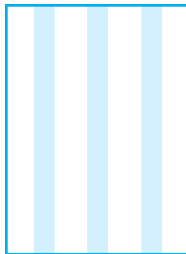
- Operations can be divided into two groups.
- One group includes set operations from mathematical set theory: these are applicable because each relation is defined to be a set of tuples in the formal relational model.



Unary Operations



(a) Selection



(b) Projection



$$\sigma_{\text{predicate}} R$$

where the symbol σ (sigma) is used to denote the SELECT operator and the condition (predicate) is a Boolean expression (condition) specified on the attributes of relation R .

- The Selection operation is used to choose a subset of the tuples from a relation that satisfies a selection condition.
- One can consider the Selection operation to be a **filter** that keeps only those tuples that satisfy a qualifying condition.
- The Selection operation can also be visualized as a horizontal partition of the relation into two sets of tuples:
 - those tuples that satisfy the condition and are selected, and
 - those tuples that do not satisfy the condition and are discarded.

Selection Example



Staff

staffNo	fName	lName	position	sex	DOB	salary	branchNo
SL21	John	White	Manager	M	1-Oct-45	30000	B005
SG37	Ann	Beech	Assistant	F	10-Nov-60	12000	B003
SG14	David	Ford	Supervisor	M	24-Mar-58	18000	B003
SA9	Mary	Howe	Assistant	F	19-Feb-70	9000	B007
SG5	Susan	Brand	Manager	F	3-Jun-40	24000	B003
SL41	Julie	Lee	Assistant	F	13-Jun-65	9000	B005

List all staff with salary greater than 10,000

$$\sigma_{\text{Salary} > 10000}(\text{Staff})$$

- Here, the input relation is Staff and the predicate is $salary > 10000$.
- More complex predicates can be generated using the logical operators $\wedge$ (AND), $\vee$ (OR) and $\neg$ (NOT).

staffNo	fName	lName	position	sex	DOB	salary	branchNo
SL21	John	White	Manager	M	1-Oct-45	30000	B005
SG37	Ann	Beech	Assistant	F	10-Nov-60	12000	B003
SG14	David	Ford	Supervisor	M	24-Mar-58	18000	B003
SG5	Susan	Brand	Manager	F	3-Jun-40	24000	B003

Figure 1: Selecting $salary > 10000$ from the Staff relation



The Selection operation can be determined as follows:

- The `<selection condition>` is applied independently to each individual tuple t in R .
- This is done by substituting each occurrence of an attribute A_i in the selection condition with its value in the tuple $t[A_i]$.
- If the condition evaluates to TRUE, then tuple t is selected.
- All the selected tuples appear in the result of the SELECT operation.



- SELECT operator is unary: applied to a single relation.
- As the selection operation is applied to each tuple individually, selection conditions *cannot* involve more than one tuple.
- The degree of the relation resulting from a SELECT operation (its number of attributes) is the same as the degree of R .
- The tuple count in the result relation is always less than or equal to the number of tuples in R , for any C :

$$|\sigma_C(R)| \leq |R|$$



- The fraction of tuples selected by a selection condition is referred to as the *selectivity* of the condition.
- Notice that **the SELECT operation is commutative** and thus, a sequence of SELECTs can be applied *in any order*.

$$\sigma_{\langle \text{cond1} \rangle} (\sigma_{\langle \text{cond2} \rangle} (R)) = \sigma_{\langle \text{cond2} \rangle} (\sigma_{\langle \text{cond1} \rangle} (R))$$



- In addition, we can always combine a cascade (or sequence) of SELECT operations into a single SELECT operation with a conjunctive (AND) condition:

$$\sigma_{\langle \text{cond1} \rangle} (\sigma_{\langle \text{cond2} \rangle} (\setminus \dots (\sigma_{\langle \text{condn} \rangle} (R)) \setminus \dots)) =$$
$$\sigma_{\langle \text{cond1} \rangle \text{ AND } \langle \text{cond2} \rangle \text{ AND } \setminus \dots \text{ AND } \langle \text{condn} \rangle} (R) \dots$$

Or we can also write as

$$\sigma_{\langle \text{cond1} \rangle \wedge \langle \text{cond2} \rangle \wedge \setminus \dots \wedge \langle \text{condn} \rangle} (R) \dots$$

Nested queries can be unnested and rewritten using AND operators.



- The PROJECT operation selects certain *columns* and discards the remaining *columns*.
- The PROJECT operation can be visualised as a vertical partition of the relation into two relations: one with the chosen columns, and the other with discarded columns.
- Produce a list of salaries for staff, showing *only* the *staffNo*, *fName*, *lName*, and *salary* details.

$$\pi_{staffNo, fName, lName, Salary}^{(STAFF)}$$



- The resulting relation is shown in Figure 2.
- The general form of the PROJECT operation is shown below, where π (π) is the symbol used to represent the PROJECT operation, and `<attribute list>` is the desired sublist of attributes from relation R .
- As with SELECT, the result is a relation.

$$\pi_{\text{<attribute list>}}(R)$$



staffNo	fName	lName	salary
SL21	John	White	30000
SG37	Ann	Beech	12000
SG14	David	Ford	18000
SA9	Mary	Howe	9000
SG5	Susan	Brand	24000
SL41	Julie	Lee	9000

Figure 2: Projecting the Staff relation over the staffNo, fName, lName, and salary attributes.



- The result of the PROJECT operation has only the attributes specified in `<attribute list>` in the same order as they appear in the list.
- Thus, its **degree** is equal to the number of attributes in `<attribute list>`.
- If the attribute list includes only nonkey attributes of R , duplicate tuples are likely to occur.
- The PROJECT operation removes any duplicate tuples, so the result of the PROJECT operation is a set of distinct tuples and thus, a valid relation.
- This formal process is reversed in SQL where the DISTINCT keyword is required to enforce the relational algebra.



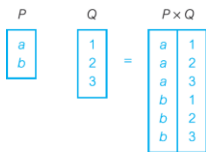
- The number of tuples in a relation resulting from a PROJECT operation is always less than or equal to the number of tuples in R .
- If the projection list is a superkey of R , ie. it includes some key of R , the resulting relation has the same number of tuples as R .
- Moreover, $\pi \langle \text{list1} \rangle (\pi \langle \text{list2} \rangle (R)) = \pi \langle \text{list1} \rangle (R)$ providing $\langle \text{list2} \rangle$ contains the attributes in $\langle \text{list1} \rangle$.
- Otherwise, the left-hand side is an incorrect expression. It is attempting to project an attribute that is not present!
- Thus, **commutativity does not hold on PROJECT**.



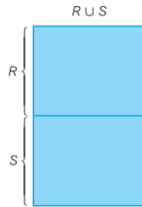
- To apply multiple algebra operations, either write a *single* relational algebra expression using *nesting*, or apply one operation at a time and create **intermediate** result relations.
- In the latter case, give names to the relations that hold the intermediate results.
- To retrieve the first name, last name, and salary of all employees who earn above 10000, apply both SELECT *and* PROJECT operations.
- Using a single relational algebra expression:

$$\pi \text{ staffNo, fName, lName, Salary } (\sigma \text{ Salary } > 10000 (\text{STAFF}))$$

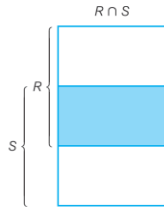
Set Operations



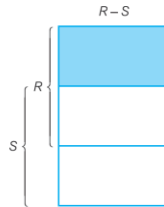
(c) Cartesian product



(d) Union



(e) Intersection



(f) Set difference

The UNION, INTERSECTION, and MINUS Operations



- The next group of relational algebra operations are the standard mathematical operations on sets.
- UNION
- INTERSECTION
- MINUS (or SET DIFFERENCE)



- Several set theoretic operations are used to merge the elements of two sets in various ways, including UNION, INTERSECTION, and SET DIFFERENCE.
- These are binary operations: each is applied to *two* sets (of tuples).
- When these operations are adapted to relational databases, the two relations on which any of these three operations are applied *must have the same type* of tuples.
- This condition has been called union compatibility or type compatibility.



$R \cup S$

The union of two relations R and S defines a relation that contains all the tuples of R , or S , or both R and S .

- If R and S have I and J tuples, respectively, their union is obtained by concatenating them into one relation with a *maximum* of $(I + J)$ tuples.
- Union is only possible where the schemas of the two relations match: the same *number* of attributes with matching attributes having the *same domain*.
- In other words, relations must be **union-compatible**.
- Attributes names are not used in defining union compatibility.
- Duplicate tuples are eliminated.

Branch

branchNo	street	city	postCode
B005	22 Deer Rd	London	SW1 4EH
B007	16 Argyll St	Aberdeen	AB2 3SU
B003	163 Main St	Glasgow	G11 9QX
B004	32 Manse Rd	Bristol	BS99 1NZ
B002	56 Clover Dr	London	NW10 6EU

PropertyForRent

propertyNo	street	city	postcode	type	rooms	rent	ownerNo	staffNo	branchNo
PA14	16 Holhead	Aberdeen	AB7 5SU	House	6	650	CO46	SA9	B007
PL94	6 Argyll St	London	NW2	Flat	4	400	CO87	SL41	B005
PG4	6 Lawrence St	Glasgow	G11 9QX	Flat	3	350	CO40		B003
PG36	2 Manor Rd	Glasgow	G32 4QX	Flat	3	375	CO93	SG37	B003
PG21	18 Dale Rd	Glasgow	G12	House	5	600	CO87	SG37	B003
PG16	5 Novar Dr	Glasgow	G12 9AX	Flat	4	450	CO93	SG14	B003

List all cities where there is either a branch office or a property for rent.

$$\pi_{city}(Branch) \cup \pi_{city}(PropertyForRent)$$



To produce union-compatible relations, use the Projection operation to project the *Branch* and *PropertyForRent* relations over the attribute *city*.

city
London
Aberdeen
Glasgow
Bristol

Figure 3: Union based on the city attribute from the Branch and PropertyForRent relations (duplicates eliminated).



R - S

The Set Difference (or MINUS) operation defines a relation consisting of the tuples that are in relation R, *but not in* S.

- As in the previous example, we produce union-compatible relations by projecting the *Branch* and *PropertyForRent* relations over the attribute *city*.
- We then use the **Set difference** operation to combine these new relations to produce the result in Figure 4.
- R and S must be union-compatible.

Set difference Operation



List all cities where there is a branch office but no properties for rent.

$$\pi_{city}(Branch) - \pi_{city}(PropertyForRent)$$

city
Bristol

Figure 4: Set difference based on the city attribute from the *Branch* and *PropertyForRent* relations.



$R \cap S$

The Intersection operation defines a relation consisting of the set of all tuples that are in both R and S .

List all cities where there is a branch office and also properties for rent.

$$\pi_{city}(Branch) \cap \pi_{city}(PropertyForRent)$$



Again, we produce union-compatible relations by projecting the *Branch* and *PropertyForRent* relations over the attribute *city*. We then use the Intersection operation to combine these new relations to produce the final result.

city
Aberdeen
London
Glasgow

Figure 5: Intersection based on city attribute from the Branch and PropertyForRent relations.



- Note that INTERSECTION can be expressed in terms of union and set difference as follows:
$$R \cap S = ((R \cup S) - (R - S)) - (S - R)$$
- In SQL, there are three operations: UNION, INTERSECT, and EXCEPT, that correspond to the set operations described here.
- In addition, there are multiset operations (UNION ALL, INTERSECT ALL, and EXCEPT ALL) that do not eliminate duplicates.



- The CARTESIAN PRODUCT operation (or CROSS PRODUCT or CROSS JOIN), is denoted by $\times$.
- This operation produces a new element by combining every member (tuple) from one relation (set) with every member (tuple) from the other relation (set).
- In general, the result of $R(A_1, A_2, \dots, A_n) \times S(B_1, B_2, \dots, B_m)$ is a relation Q with degree $n + m$ attributes $Q(A_1, A_2, \dots, A_n, B_1, B_2, \dots, B_m)$, in that order.
- The resulting relation Q has one tuple for each *combination of tuples*: one from R and one from S .
- Thus, if R has n_R tuples (denoted as $|R| = n_R$), and S has n_S tuples, then $R \times S$ will have $n_R * n_S$ tuples.



- In general, the CARTESIAN PRODUCT operation applied by itself is generally meaningless.
- It is mostly useful when followed by a selection that matches values of attributes coming from the component relations.
- Select all books, *loaned to a particular student*.
- Select all staff, *who work in a particular branch*.



List the names and comments of all clients who have viewed a property for rent.

- The names of clients are held in the *Client* relation and the details of viewings are held in the *Viewing* relation.
- To obtain the list of clients and the comments on properties they have viewed, we need to combine these two relations:

$$\pi(\text{clientNo}, \text{fName}, \text{lName})^{(Client)} \times (\pi \text{ clientNo}, \text{propertyNo}, \text{comment})^{(Viewing)}$$

Cartesian Product Result

In its present form, this relation contains more information than we require. For example, the first tuple of this relation contains different *clientNo* values.

client.clientNo	fName	IName	Viewing.clientNo	propertyNo	comment
CR76	John	Kay	CR56	PA14	too small
CR76	John	Kay	CR76	PG4	too remote
CR76	John	Kay	CR56	PG4	
CR76	John	Kay	CR62	PA14	no dining room
CR76	John	Kay	CR56	PG36	
CR56	Aline	Stewart	CR56	PA14	too small
CR56	Aline	Stewart	CR76	PG4	too remote
CR56	Aline	Stewart	CR56	PG4	
CR56	Aline	Stewart	CR62	PA14	no dining room
CR56	Aline	Stewart	CR56	PG36	
CR74	Mike	Ritchie	CR56	PA14	too small
CR74	Mike	Ritchie	CR76	PG4	too remote
CR74	Mike	Ritchie	CR56	PG4	
CR74	Mike	Ritchie	CR62	PA14	no dining room
CR74	Mike	Ritchie	CR56	PG36	
CR62	Mary	Tregear	CR56	PA14	too small
CR62	Mary	Tregear	CR76	PG4	too remote
CR62	Mary	Tregear	CR56	PG4	
CR62	Mary	Tregear	CR62	PA14	no dining room
CR62	Mary	Tregear	CR56	PG36	

Cartesian Product Result (Final)

To finalise, use a Selection on this relation to extract those tuples where *Client.clientNo* = *Viewing.clientNo*.

$$\sigma_{Client.clientNo = Viewing.clientNo}((\pi_{clientNo, fName, lName}^{(Client)}) \times (\pi_{clientNo, propertyNo, comment}^{(Viewing)}))$$

client.clientNo	fName	lName	Viewing.clientNo	propertyNo	comment
CR76	John	Kay	CR76	PG4	too remote
CR56	Aline	Stewart	CR56	PA14	too small
CR56	Aline	Stewart	CR56	PG4	
CR56	Aline	Stewart	CR56	PG36	
CR62	Mary	Tregear	CR62	PA14	no dining room

Figure 7: Restricted Cartesian product of reduced Client and Viewing relations.



- Relational algebra operations can be arbitrarily complex.
- Decompose such operations into a series of smaller relational algebra operations and give a name to the results of intermediate expressions.
- We use the assignment operation, denoted by $\leftarrow$, to name the results of a relational algebra operation.

TempViewing(*clientNo*, *propertyNo*, *comment*)

$\leftarrow \pi$ *clientNo*, *propertyNo*, *comment*

(*Viewing*) *TempClient*(*clientNo*, *fName*, *lName*)

$\leftarrow \pi$ *clientNo*, *fName*, *lName*^(*Client*)

Comment(*clientNo*, *fName*, *lName*, *vclientNo*, *propertyNo*, *comment*)

$\leftarrow$ *TempClient* $\times$ *TempViewing*

Result $\leftarrow \sigma$ _{*clientNo* = *vclientNo*}^(*Comment*)



- The result of the JOIN is a relation Q with $n + m$ attributes $Q(A_1, A_2, \dots, A_n, B_1, B_2, \dots, B_m)$ in that order.
- Q has one tuple for each combination of tuples, one from R and one from S , whenever the combination *satisfies the join condition*.
- This is the main difference between CARTESIAN PRODUCT and JOIN.
- In JOIN, only combinations of tuples *satisfying the join condition* appear in the result; in CARTESIAN PRODUCT *all combinations* of tuples are included in the result.

Join Operations Illustrated

$$T$$

A	B
a	1
b	2

$$U$$

B	C
1	x
1	y
3	z

$$T \bowtie U$$

A	B	C
a	1	x
a	1	y

(g) Natural join

$$T \triangleright_B U$$

A	B
a	1

(h) Semijoin

$$T \bowtie_C U$$

A	B	C
a	1	x
a	1	y
b	2	

(i) Left Outer join

θ -join

theta join defines a relation containing tuples satisfying the predicate F from the Cartesian product of R and S .

$$R \bowtie_F S = \sigma_F(R \times S)$$

- The predicate F is of the form $R.a_i \theta S.b_i$ where θ is a comparison operator.
- As with Cartesian product, the **degree** of a theta join is the **sum of the degrees** of relations R and S .
- We can rewrite the theta join in terms of the basic Selection and Cartesian product operations:

In the case where the predicate F contains only equality ($=$), the term **Equijoin** is used instead.

List the names and comments of all clients who have viewed a property for rent.

Previously, we used the Cartesian product and Selection for this. The same result is obtained using the Equijoin operation:

$$(\pi \text{ clientNo, fName, lName}^{(Client)}) \bowtie_{Client.clientNo = Viewing.clientNo} (\pi \text{ clientNo, propertyNo, comment}^{(Viewing)})$$

or do the projections first and:

$$Result \leftarrow TempClient \bowtie_{Client.clientNo = Viewing.clientNo} (TempViewing)$$

The result of these operations was shown in Figure 7.



$R \bowtie S$

The Natural join is an Equijoin of the two relations R and S over all common attributes x .

- One occurrence of each common attribute is eliminated from the result.
- The **Natural Join** operation performs an **Equijoin** over all the attributes in the two relations *that have the same name*.
- The degree of a natural join is the *sum of the degrees* of the relations R and S less the number of attributes in x .
- There is no need to duplicate columns, unlike Cartesian Product with condition on slide 2.36!

Natural Join Example



List the names and comments of all clients who have viewed a property for rent.

In figure 7, the Equijoin produced the result, but the resulting relation had two occurrences of (join attribute) *clientNo*. Use the Natural join to reduce to one *clientNo* column.

$$\pi \text{ clientNo, fName, lName}^{(Client)} \bowtie (\pi \text{ clientNo, propertyNo, comment}^{(Viewing)})$$

or

$$\text{Result} \leftarrow \text{TempClient} \bowtie \text{TempViewing}$$

Natural Join Result

clientNo	fName	IName	propertyNo	comment
CR76	John	Kay	PG4	too remote
CR56	Aline	Stewart	PA14	too small
CR56	Aline	Stewart	PG4	
CR56	Aline	Stewart	PG36	
CR62	Mary	Tregear	PA14	no dining room

Figure 8: Natural join of restricted Client and Viewing relations.



$R \bowtie S$

The (left) Outer join is a join in which tuples from R that do not have matching values in the common attributes of S are also included in the result relation.

- Missing values in the second relation are set to null.
- The advantage of an Outer join is that information is preserved.
- Outer join preserves tuples that would have been lost by other types of join, e.g. Natural Join.



Produce a status report on property viewings.

- A report is required to include all properties and their viewings, even if there were no viewings.
- The advantage of an Outer join is that information is preserved.
- Outer join preserves tuples that would have been lost by other types of join, e.g. Natural Join.

The target is a relation consisting of the properties that have been viewed with comments and those that have not been viewed.

$$\pi \text{ propertyNo, street, city}(\text{PropertyForRent}) \bowtie \text{Viewing}$$

Outer Join Result

Note that properties PL94, PG21, and PG16 have no viewings, but these tuples are still contained in the result with nulls for the attributes from the *Viewing* relation.

propertyNo	street	city	clientNo	viewDate	comment
PA14	16 Holhead	Aberdeen	CR56	24-May-04	too small
PA14	16 Holhead	Aberdeen	CR62	14-May-04	no dining room
PL94	6 Argyll St	London	null	null	null
PG4	6 Lawrence St	Glasgow	CR76	20-Apr-04	too remote
PG4	6 Lawrence St	Glasgow	CR56	26-May-04	
PG36	2 Manor Rd	Glasgow	CR56	28-Apr-04	
PG21	18 Dale Rd	Glasgow	null	null	null
PG16	5 Novar Dr	Glasgow	null	null	null

In effect, this is a Left (natural) Outer join as it keeps every tuple in the left-hand relation in the result.

Similarly, a Right Outer join ($R \bowtie S$) retains every tuple from the right-hand relation in the result.

Lecture Ends ...

[Relational Algebra](#)



[The Relational
Algebra: An Overview](#)

[Unary Operations](#)

[Set Operations](#)

[Join Operations](#)

